

御网智元技术报告

具备自主决策能力的通用网络安全智能体

版本日期：2026-09-03

项目摘要

一句话介绍

御网智元是一个面向授权附件和本地靶场的、可审计的通用网络安全智能体工作台。它将自然语言任务转换为可执行计划，按授权范围选择受控工具，保存公开行动轨迹，并用独立验证器区分“模型判断”“工具事实”和“已验证结论”。

解决的问题

传统安全自动化通常有两类缺陷：固定脚本只能处理预设路径；聊天式模型能够给建议，却不能稳定完成工具调用、失败调整和结果验证。御网智元将任务理解、规划、执行、观察、重规划、验证和报告固化为一个可恢复的 Run 状态机，重点解决以下问题：

- 复杂安全任务如何拆分为可执行步骤；
- 工具失败、越权或证据不足时如何安全调整路径；
- 如何让每一步行动都有公开理由和可追溯来源；
- 如何避免模型把自述文本伪装成事实或验证结果；
- 如何在单机环境中完成部署、审计、恢复和评测。

作品能力

- 多场景任务入口：CTF、应急响应、漏洞分析、静态逆向和通用研判；
- 默认动态 Agent：先生成 Task Brief 和计划，再执行受控工具并根据观察重规划；
- 统一工具契约：ToolSpec、Registry、PolicyEngine、Executor 和 MCP 扩展边界；
- 离线安全分析：文件元数据、文本定位、IOC、时间线、编码、哈希、JWT、PCAP 摘要等；
- 可信结果闭环：Artifact、ToolCall、EvidenceRecord、TaskResult 和独立 Judge；

- 运行可靠性：SQLite 审计、检查点恢复、上下文压缩、预算和幂等控制；
- 工程交付：Docker Compose、Windows 启动脚本、健康检查、诊断、备份和恢复。

可核验边界

当前版本不提供任意 Shell、公网扫描、自动攻击、多智能体协作或外部 CTF 平台提交。网络演示限定在用户明确授权的本机目标，附件和工具输出均按不可信数据处理。真实 Provider 成绩只有在有可复查脱敏记录时统计，测试替身不冒充真实成绩。

技术方案

1. 设计目标

作品围绕赛题要求的自主任务理解、多场景决策执行、可解释可审计和稳定运行展开。每次运行形成一条可复查事实链：任务输入 → 计划 → 受控行动 → 工具观察 → 路径调整 → 结果候选 → 独立验证 → 报告。

2. 运行流程

1. 用户提交自然语言目标、场景、授权目标和附件。
2. 系统创建不可变 TaskSpec 和 Run，冻结 Provider、Agent Profile、工具清单、授权范围、预算及附件摘要。
3. 默认 Agent 生成公开 Task Brief，识别目标、约束、成功条件和待确认信息。
4. Planner 输出结构化计划，每步包含目标、理由、预期结果和验证方式。
5. ActionSelector 选择 `call\_tool`、`replan`、`finish`、`fail` 或 `request\_input`。
6. PolicyEngine 检查工具、参数、目标范围、风险、权限、预算和附件信任等级。
7. Executor 执行已注册工具，持久化 ToolCall、ExecutionStep、Artifact 和公开事件。

8. Agent 读取确定性观察；工具失败、无进展或新证据出现时触发重规划。
9. Verify 节点根据任务规则、Evidence 和结构化 Schema 生成服务端验证状态。
10. Finalizer 写入 TaskResult，生成 Markdown/JSON 报告、复盘摘要和可回放轨迹。

3. 状态与可信度

unverified 表示没有可绑定证据；partial 表示已绑定真实工具来源但结论尚未独立验证；validated 只表示 Evidence/Judge 等服务端规则验证通过；failed 表示验证失败或 Run 未成功完成。成功工具调用不会直接等同于结论正确。

4. 输入与附件

附件通过 Artifact ID 访问，不接受模型提供的宿主机路径。上传阶段检查大小、扩展名、MIME 和 magic bytes；ZIP/TAR 进行文件数、单文件、总大小和压缩比限制。文本、JSON、YAML、CSV、归档和常见安全分析材料以有界摘要进入上下文，原始文件保留在隔离的 Artifact 存储中。

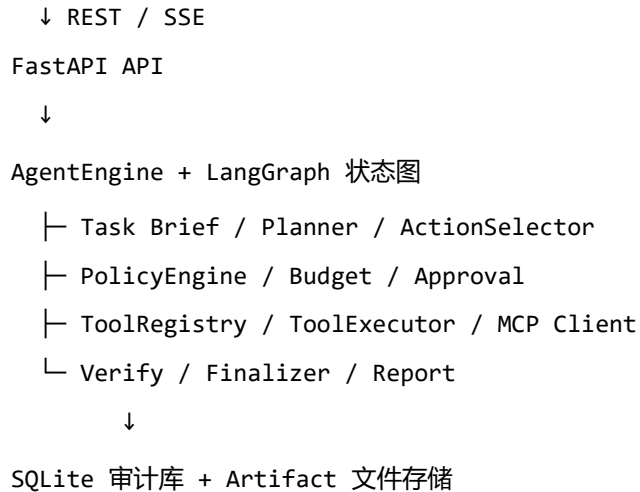
5. 当前能力与后续适配

当前已实现文本和归档安全摘要、localhost 只读 HTTP、内容定位、IOC、时间线、编码、哈希、JWT、PCAP、字符串和静态分析工具。工具通过 Python 插件或受控 MCP 扩展。决赛平台、指定模型网关和外部判分接口尚未开放，因此本版本只保留清晰适配边界，不虚构平台成绩。

系统架构与模块说明

1. 总体架构

React Web



2. 模块职责

模块	作用	关键位置
API 层	任务、消息、运行控制、报告和评测接口	apps/api/routes/
Agent 层	计划、动作、观察、重规划、验证、恢复	src/yuwang/agent/
控制模型	Task Brief、Plan、Action 和结构化输出契约	src/yuwang/control/
策略层	授权目标、权限、风险、预算和输入信任边界	src/yuwang/policy/
工具层	ToolSpec、注册、选择、执行、插件和 MCP	src/yuwang/tooling/
存储层	Run、事件、ToolCall、Artifact、Evidence、报告和检查点	src/yuwang/storage/
结果层	TaskResult、证据绑定和服务端验证状态	src/yuwang/results/
评测层	本地任务包、私有 Judge、评分和结果导出	src/yuwang/evaluation/
前端层	场景入口、任务时间线、活动流、审计和设置	apps/web/src/

3. 关键工程约束

- Run 创建后冻结 TaskSpec、Provider、Profile、工具和授权快照；
- 工具不能绕过 Registry、PolicyEngine 和 Executor；
- 外部内容只作为不可信数据进入上下文，不能提升为系统指令；
- 非幂等或状态不确定调用不会被恢复逻辑自动重放；
- 原始审计事实与展示摘要分离，报告和 UI 只读取持久化事实。

自主决策与可解释性设计

模型只输出结构化计划/动作，服务端记录公开的目标、行动理由、工具参数摘要、观察事实、下一步决策和证据引用，不保存隐藏思维链。每个动作都有策略判定和状态事件；无进展、预算、超时、越权和不可信输出会触发停止或重规划。

可复查证据：Run events、ExecutionStep、EvidenceRecord、TaskResult、Markdown/JSON 报告；恢复从持久化检查点继续，不重放已完成调用。

工具协同与扩展机制

所有工具遵循 ToolSpec、输入/输出 Schema、风险、权限、超时、错误码和幂等契约，统一经 Registry、PolicyEngine、Executor 执行。内置 CTF 工具可串联 Artifact；MCP 仅允许管理员白名单命令或受控 HTTPS，禁止 Shell 解释器。

Run 启动时会依据任务场景、工具 capabilities 和已有 Artifact 类型生成候选工具清单，再由 Agent 自主选择并记录公开理由；新增 consumes、produces、前置条件与失败替代能力不会改变这一决策边界。

新增工具需实现 SDK Plugin、补契约/越权/失败测试，并声明可处理的 Artifact 类型；Run 启动时冻结工具快照。

安全设计与威胁模型

威胁包括路径穿越、符号链接、压缩炸弹、超大文本、MIME 伪装、Prompt Injection、SSRF、密钥泄漏和恢复重放。防护包括 Artifact ID 隔离、ZIP/TAR 文件数/单文件/总大小/压缩比限制、magic bytes 与扩展名/MIME 一致性、localhost 白名单、递归脱敏、CSRF、快照和幂等请求。

公网部署不能裸露本应用：需 HTTPS、带认证反向代理、临时账号、限流、隔离数据库/Artifact 目录和凭据轮换。

创新点与同类方案对比

核心创新是把可信验证、证据绑定、不可变快照、安全恢复和不可信工具输出处理做成可审计契约，而不是把模型大小或框架名称当创新。当前版本的创新价值集中在“模型负责提出候选，服务端负责约束、执行、取证和验证”的职责分离。

公开工作对比（访问日期 2026-08-28；以下是公开仓库/论文描述，不是本项目的性能基线）：

工作	可核查来源与事实	本项目的边界差异
NYU CTF Bench	[GitHub](https://github.com/NYU-LLM-CTF/NYU_CTF_Bench)，公开 README 描述 200 个 CSAW CTF、6 类题型，并提供 Docker 化挑战	本项目当前只提供小型本地评测集，重点是证据、权限和可审计闭环，不声称覆盖其规模
BoxPwnr	[GitHub](https://github.com/Oca/BoxPwnr)、[traces](https://boxpwnr.info/)，支持多个安全平台/solver 并公开轨迹	本项目不接入公网平台，轨迹只保存脱敏公开行动和证据引用，不保存隐藏思维链
HackSynth	[GitHub](https://github.com/aiette-research/HackSynth)、[论	本项目的创新主张不是 Planner 数量，而是不可变快照、工具策略检查

	文](https://arxiv.org/abs/2412.01778) ，公开介绍 Planner/Summarizer 与 PicoCTF/OverTheWire 基准	和确定性验证
D-CIPHER	[NYU agents 仓 库](https://github.com/NYU-LLM- CTF/nyuctf_agents)中包含 D-CIPHER 相关实现/说明	当前不复现其结果，不将名称相同视 为性能或方法等价
CyberGym-E2E	[GitHub](https://github.com/sunblaz e-ucb/cybergym-e2e)、[论 文](https://arxiv.org/abs/2606.04460) ，公开描述从漏洞发现、PoC 到补丁 的端到端评测	本项目不执行真实项目漏洞利用或补 丁评测，案例限定无害 localhost/附 件

本项目没有足够同口径数据宣称优于上述工作；初审只展示可复查的本地运行记录、失败记录和安全边界。与大规模 CTF 基准相比，本项目规模较小，但更强调权限、证据和失败可恢复性。

御网智元方案设计文档

1. 设计原则

方案以“自主但受控、可解释且可复现、结果必须有证据”为三条主线。模型可以提出计划和候选行动，但不能自行扩大授权、绕过工具策略或宣布未经验证的结论。

2. 场景设计

场景	输入	主要工具	交付结果
CTF	题目文本、附件、本机靶场	文件、编码、哈希、Flag、localhost HTTP	线索链、候选结果、验证边界
应急响应	日志、流量、IOC 材料	IOC、时间线、PCAP、内容搜索	事件时间线、指标和处置建议

漏洞分析	源码、接口说明、响应材料	Web 证据、源码模式、HTTP 只读分析	风险判断、复现步骤和修复建议
静态逆向	字符串、二进制元数据	字符串、文件、静态元数据	结构线索和下一步分析计划

3. 自主决策设计

任务开始先生成 Task Brief，再生成计划。每个计划步骤必须有目标、理由、预期结果和验证方法。执行后只将结构化工具观察回传给模型；当工具失败、路径无进展、证据不足或出现新线索时，Agent 选择重规划、请求补充信息或安全结束。

4. 人机协同设计

用户可以在运行中追加指引、暂停、继续、停止或从检查点恢复。高风险动作需要审批；当前默认工具以离线分析和授权 localhost 只读探测为主。人机交接摘要会列出已验证结果、未验证候选、当前阻塞和建议接手动作。

5. 结果设计

模型输出只包含结果类型、标题、摘要、结构化数据、证据候选和置信度。服务端从当前 Run 绑定真实 ToolCall、Artifact 或 EvidenceRecord，并独立写入验证状态。报告、前端结果卡、评测和轨迹回放都读取同一份持久化结果。

6. 评测设计

本地评测将输入、允许工具、授权范围、预算、结构化结果 Schema 和私有 Judge 分离。评测检查工具是否调用、证据是否来自正确工具、是否越权、是否超预算以及结构化结果是否正确，不把非空文本或模型自述当作成绩。

7. 终审适配

主办方决赛平台开放后，优先替换 Provider 网关、目标授权适配器和外部结果提交接口；现有 Run、ToolSpec、Evidence、轨迹和评测接口保持不变。当前版本不虚构尚未提供的平台能力。

开发文档

1. 技术栈

- 后端：Python 3.11、FastAPI、Pydantic、LangGraph、SQLite；
- 前端：React、TypeScript、Vite；
- 部署：Docker Compose、Nginx、Docker Desktop；
- 扩展：声明式 Skill、Python Plugin、受控 MCP；
- 质量：pytest、Ruff、mypy、Vitest、TypeScript、生产构建和浏览器验收。

2. 代码组织

后端业务代码位于 `src/yuwang`，API 装配位于 `apps/api`，前端位于 `apps/web`，本地评测任务位于 `evaluation_cases`，工具沙箱位于 `tool_sandbox`。新增工具应实现统一输入/输出模型和 ToolSpec，注册到 Registry，并补齐契约、越权、失败和脱敏测试。

3. 本地开发

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
cd apps/web
npm ci
cd ../..
```

配置 .env 时只使用本机 Provider 和随机管理密钥；.env、data/ 和日志不进入 Git。首次运行：

```
.\yuwang.ps1 setup
.\yuwang.ps1 start
.\yuwang.ps1 doctor
```

4. 变更流程

11. 先阅读对应模块和测试，再修改最小范围代码；
12. 为公开行为补单元或 API 测试；
13. 工具变更补 Schema、权限、授权范围、错误码和输出脱敏检查；
14. 运行 `ruff check src tests`、`mypy` 和相关 pytest；
15. 提交前运行 `.\yuwang.ps1 check`、前端质量门禁和 Docker 验收；
16. 检查 `git diff --check` 与 `git status`，确认没有密钥、数据库和缓存。

5. Provider 与模型适配

Provider 通过统一结构化输出适配器接入。模型只负责公开计划、动作理由和结构化候选，服务端负责策略、执行、证据绑定和验证。真实 Provider 测试需配置隔离密钥，不能用测试替身结果冒充实测成绩。

6. 扩展边界

允许增加离线解析、授权本机探测和有界取证工具；不在本阶段引入任意 Shell、公网扫描、自动攻击或多租户。MCP 接入必须管理员显式白名单、声明权限和目标范围，并经过同一策略与审计链。